Open in app ↗ Sign up Sign In

Published in Towards Data Science

Thomas Albin Follow

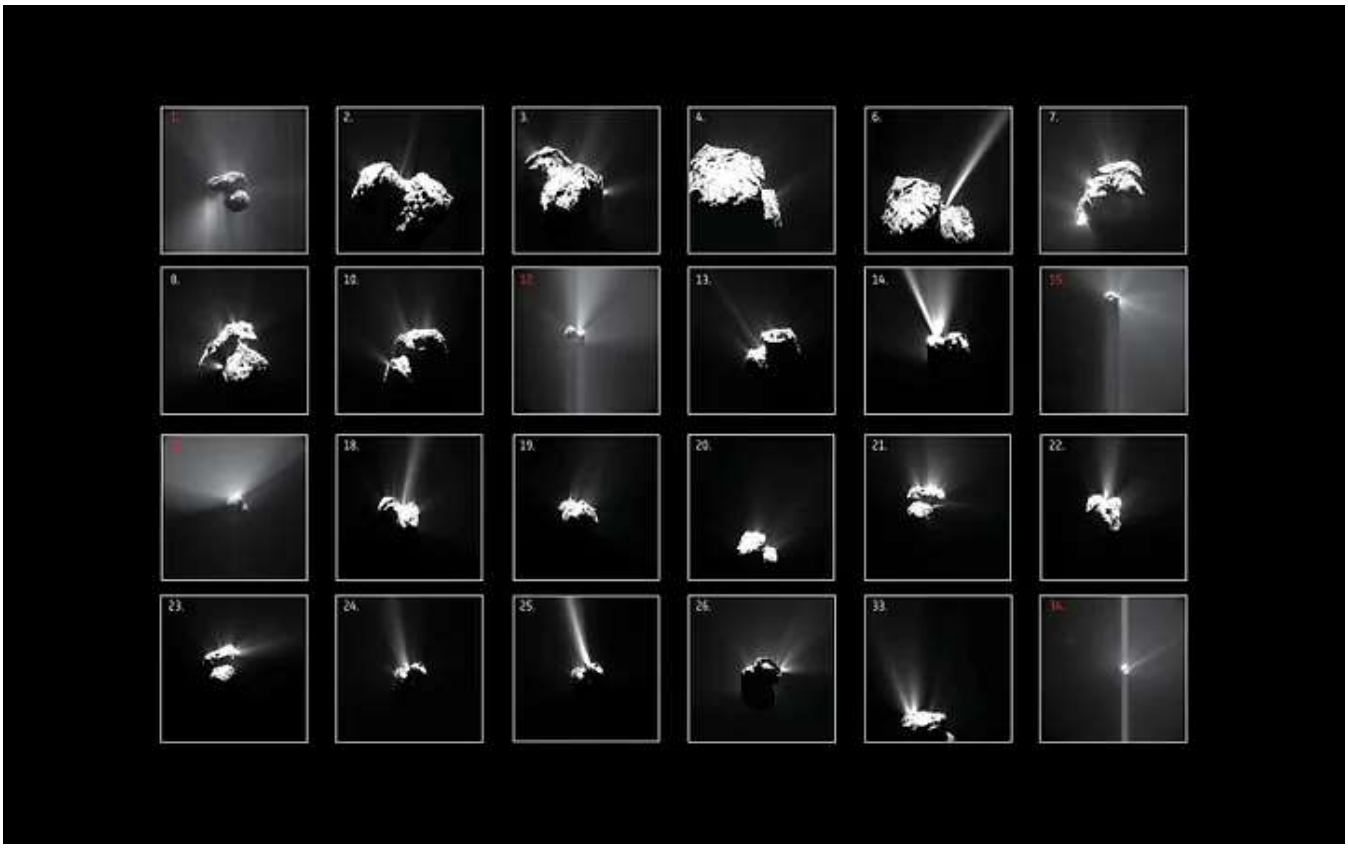
Jun 6, 2020 · 9 min read · ✨ · Listen

Save

SPACE SCIENCE WITH PYTHON

Space Science with Python — Turbulent times of a comet

Part 13 of the tutorial series will take a look at the orbital elements of 67P/Churyumov–Gerasimenko. It is simple orbital mechanics. Right?



The different faces of 67P/Churyumov–Gerasimenko as seen from the Rosetta spacecraft. Credit: ESA/Rosetta/MPS for OSIRIS Team MPS/UPD/LAM/IAA/SSO/INTA/UPM/DASP/IDA; NavCam: ESA/Rosetta/N[illegible] BY-SA IGO 3.0

8

Preface

This is the 13th part of my Python tutorial series "Space Science with Python". All codes that are shown here are uploaded on GitHub. Enjoy!

Users suggested adding small tasks at the end of every Jupyter Notebook to solve scientific or programming questions. Starting with this session I will add these assignments and provide a solution after some time on GitHub. Have fun!

Introduction

In the last session (Space Science with Python — A comet in 3 D) we started to take look at a very particular comet: 67P/Churyumov–Gerasimenko. A comet that was visited by ESA's Rosetta / Philae mission. From 2014 to 2016 the orbiter explored this icy world and deployed its lander Philae in November 2014. A few hours after approaching the comet humanity gathered more data and insights about comets than all combined missions and observations before. In 2004 a book called *Comets 2* was published that summarised all relevant information about these evaporating worlds [1]. Now, we have more scientific findings that are partly freely available online: *Astronomy & Astrophysics Special Issues*.

Citizen scientists contribute great content for the scientific and public community. This image of the comet's surface is a collection of several images that were merged by an amateur astronomer. The official ESA image description says: "[...] Amateur astronomer Jacint Roger Perez, from Spain, selected and processed this view by combining three images taken in different wavelengths by the OSIRIS narrow-angle camera on Rosetta. [...]". Credit: ESA/Rosetta/MPS for OSIRIS Team MPS/UPD/LAM/IAA/SSO/INTA/UPM/DASP/IDA; J. Roger; License: CC BY SA 4.0

Our last session gave us a detailed overview of the 3 D structure of the comet that was derived by one of Rosetta's camera systems. Today and in our next session we want to learn more about its trajectory; its movement around the Sun. *How does the comet revolve around the Sun? Are there different information resources? And what kind of insights can we derive from it?*

Nothing to expect, right?

Considering simple 2 body problems, orbital mechanics are quite simple to understand. Two masses are revolving around a common centre of mass. In a previous session (Space Science with Python: Comets — Visitors from afar) we created an SQLite database that contains the orbital elements of all known comets. The data were obtained from the Minor Planet Center (MPC). These data can be used to compute the trajectory of the comet. *But is it really that simple?*

First, we import our required modules. This time, we will need NASA's SPICE toolkit respectively the Python wrapper *spiceypy*. Other modules, like *numpy*, *pandas* and *matplotlib* are also being used.

Part 1/8

We establish a connection to our database in line 2. Due to its small size of around 180 kB the database is also stored on my GitHub repository, so you do not need to worry to create this database from scratch.

What kind of data do we want to describe the trajectory of 67P? Well, obviously we need the orbital elements like the perihelion (line 5), the semi-major axis (line 6), the aphelion and eccentricity (line 7) as well as the argument of perihelion and the inclination (line 8). The comets' names start with the number and type abbreviation (C, P, X etc.) followed by the actual name. To get the data of a comet by only providing the number we use the SQL command *NAME LIKE "67P%"*. The *where* statement searches for names whose strings are "like" 67P with any (sub) string attached (%). The data are stored in a *pandas* dataframe via the command *read_sql*.

Part 2/8

Let's take a look at the data from the database. We simply print the results with the *iloc* command within an f-string.

Part 3/8

The output is shown below. We successfully extracted the information of the comet 67P. The perihelion is at 1.2 AU, so within the range where the Sun's heating causes the comets to evaporate [2]. The aphelion is at 5.7 AU; and a small inclination of around 4 degrees indicates that the comet crosses the orbit's plane of Jupiter, making it a Jupiter-Family-Comet (JFC).

```
NAME                  67P/Churyumov-Gerasimenko
PERIHELION_AU                           1.20213
SEMI_MAJOR_AXIS_AU                      3.45513
APHELION_AU                             5.70813
ECCENTRICITY                           0.652073
ARG_OF_PERIH_DEG                        21.8178
INCLINATION_DEG                          3.8868
Name: 0, dtype: object
```

With these data, we could easily visualise and analyse the comet's orbit. But consider the article about the Tisserand Parameter (Space Science with Python — A Rendezvous with Jupiter): The orbit of small bodies is affected by planetary gravitational perturbations (like Jupiter). Perturbations alter the orbital elements, however, the database provides only **static results** at a certain epoch!

So shall we generate complex and computational heavy numerical simulations? This is not necessary. Re-consider the already used SPICE kernels. Maybe we find a kernel that already contains some more complex considerations. Let's have a look.

The public kernel repository has a sub-folder for the *Rosetta* mission. There we find the *kernels* and then *spk* sub-folders. We take a deeper look at the "old" data in the sub-folder *former_versions*. The *aareadme.txt* states:

```
Name                                 Comments
---------------------------------    ---------------------------------
67P_CHURY_GERAS_2004_2016.BSP        Contains ephemeris data for the
                                     Comet Churyumov-Gerasimenko/67P.
                                     This file has been originated

from
                                     JPL Horizons On-Line Ephemeris
                                     System and IT IS NOT an official
                                     Rosetta Project SPK. It is
                                     provided as support to study
                                     cases until an official SPK is
                                     released. Spans from January 2004
                                     to January 2016.
```

Although this is not an official project *spk*, we use this kernel due to the time coverage between 2004 and 2016. We add the kernel's path (with some other needed kernels) to the *kernel_meta.txt* and load the file via *furnsh*.

For the orbital elements that were loaded before, we will need the gravitational constant times the mass of the Sun. We extract this information in line 5 and 6 with the command *bodvcd*.

Part 4/8

What do we want to achieve? The MPC data provide orbital elements that do not change over time. The SPICE kernel, on the other hand, could contain a more sophisticated dataset of the comet's trajectory. So let's analyse a time range of 10 years; from 2004 to 2014. First, we need to create an array that contains date-time steps between both years. Line 2 and 3 create *pandas Timestamp*(s) for the 1st January of each year. Line 6 creates a *numpy array* with 1000 time steps and line 9 converts it back to a date-time object with *pandas to_datetime*.

Part 5/8

Now we compute the orbital elements of 67P for each time step, based on the corresponding SPICE kernel. An empty data-frame is created in line 2 that will store all results. The UTC time strings are stored in line 5 and converted to Ephemeris time (ET) in line 8 to 9 with the SPICE function *utc2et*. Then, the state vector of 67P as seen from the Sun is computed (line 12 to 16). For this computation the function *spkgeo* is used and ECLIPJ2000 is set as a reference frame. The NAIF ID code for 67P is 1000012 [*]. Finally, line 19 to 23 calculate the state vectors' corresponding orbital elements with the function *oscltx*.

The orbital elements are stored as arrays in the column STATE_VEC_ORB_ELEM. Now, we extract the required orbital elements from the array and assign them to individual dataframe columns. The order of the orbital elements in the arrays is explained in the *oscltx* documentation:

```
elts        are equivalent conic elements describing the orbit
            of the body around its primary. The elements are,
            in order:

              RP      Perifocal distance.
              ECC     Eccentricity.
              INC     Inclination.
              LNODE   Longitude of the ascending node.
              ARGP    Argument of periapsis.
              M0      Mean anomaly at epoch.
              T0      Epoch.
              MU      Gravitational parameter.
              NU      True anomaly at epoch.
              A       Semi-major axis. A is set to zero if
                      it is not computable.
              TAU     Orbital period. Applicable only for
                      elliptical orbits. Set to zero otherwise.
```

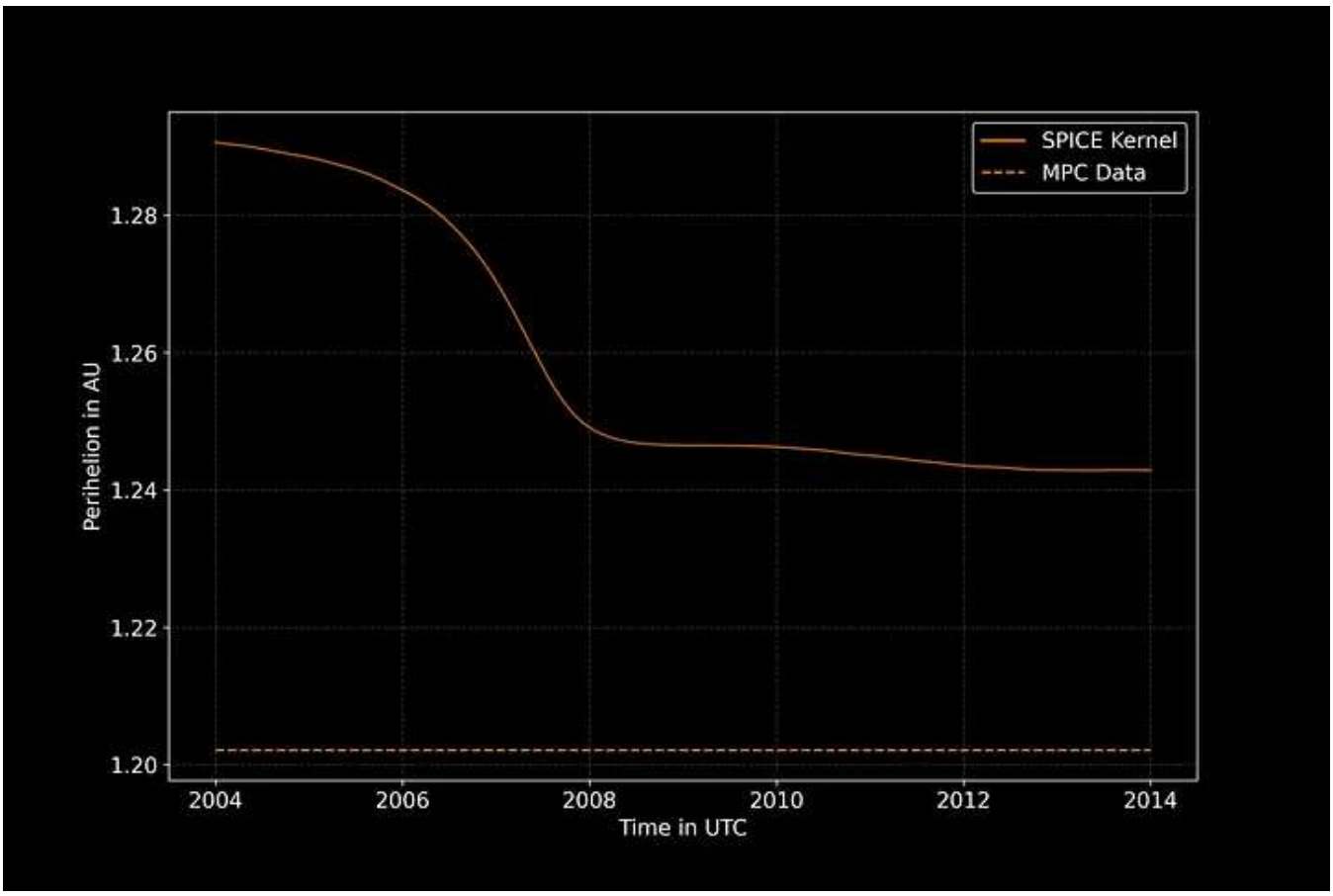
Spatial information are given in km and angular information are given in radians. We convert the values to Astronomical Units (using the function *convrt* in line 3 to 7 and 26 to 30) and degrees (using *numpy degrees* in line 14–15, 18–19 and 22–23), respectively. In lines 33 to 36 we compute additionally the aphelion using the semi-major axis and eccentricity.

Part 7/8

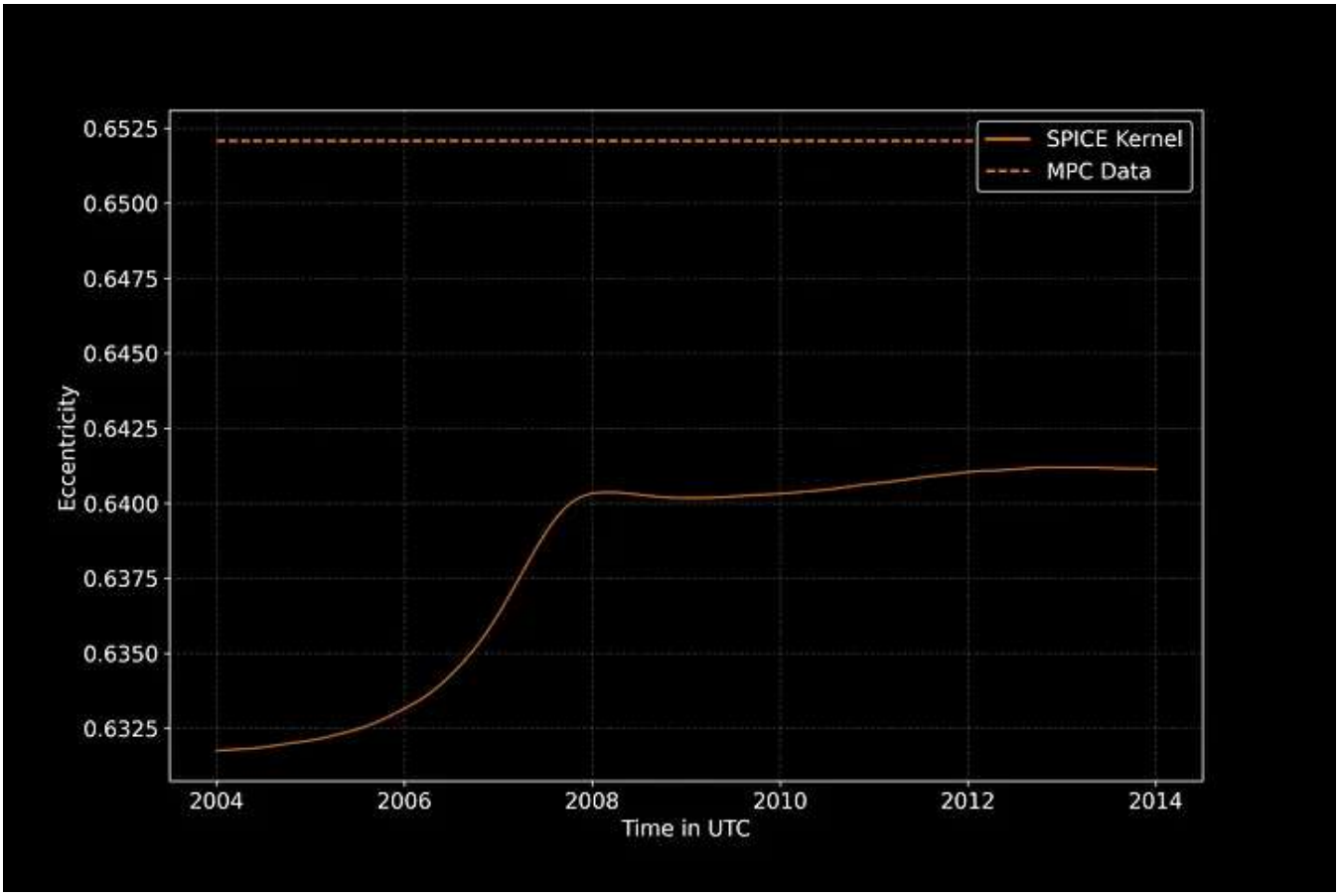
The results from the SPICE kernel and the static dataset from the MPC dataset can now be displayed. We use *matplotlib* to visualise and interpret the results. We set a dark-theme in line 4 and increase the font size for better readability in line 7. A for-loop iterates now through 3 different parameters: the perihelion, the eccentricity and the argument of perihelion (line 11 to 50). The SPICE data are plotted in line 22 to 24 and the MPC data are plotted as a horizontal reference line in code line 28 to 29 using the *matplotlib* function *hlines*. The plotting routine requires a static y value and the required x axis range (xmin and xmax). A grid (line 35) and axes labels are set (line 38 and 39). Line 43 to 47 adjust the legend settings (the transparency of the lines is being removed for better readability) and the figure is finally stored (line 50).

Part 8/8

Let's take a look at the resulting plots. The first two figures show the perihelion and eccentricity vs. the date-time given in UTC. The solid line represents the values from the SPICE kernel and the horizontal dashed line is the reference data from the MPC dataset. As you can see, both results differ. Further, the SPICE results change over time. For example, between 2004 and 2008 the perihelion changes from around 1.29 AU to less than 1.25 AU, so around 6 Million kilometres! The smallest difference between the MPC data and the SPICE kernel result is around 0.04 AU for the perihelion data and 0.0125 for the eccentricity. Considering that we did not take an error into account from the MPC data (SPICE kernel did not provide an error for the values) the difference is not significantly large.

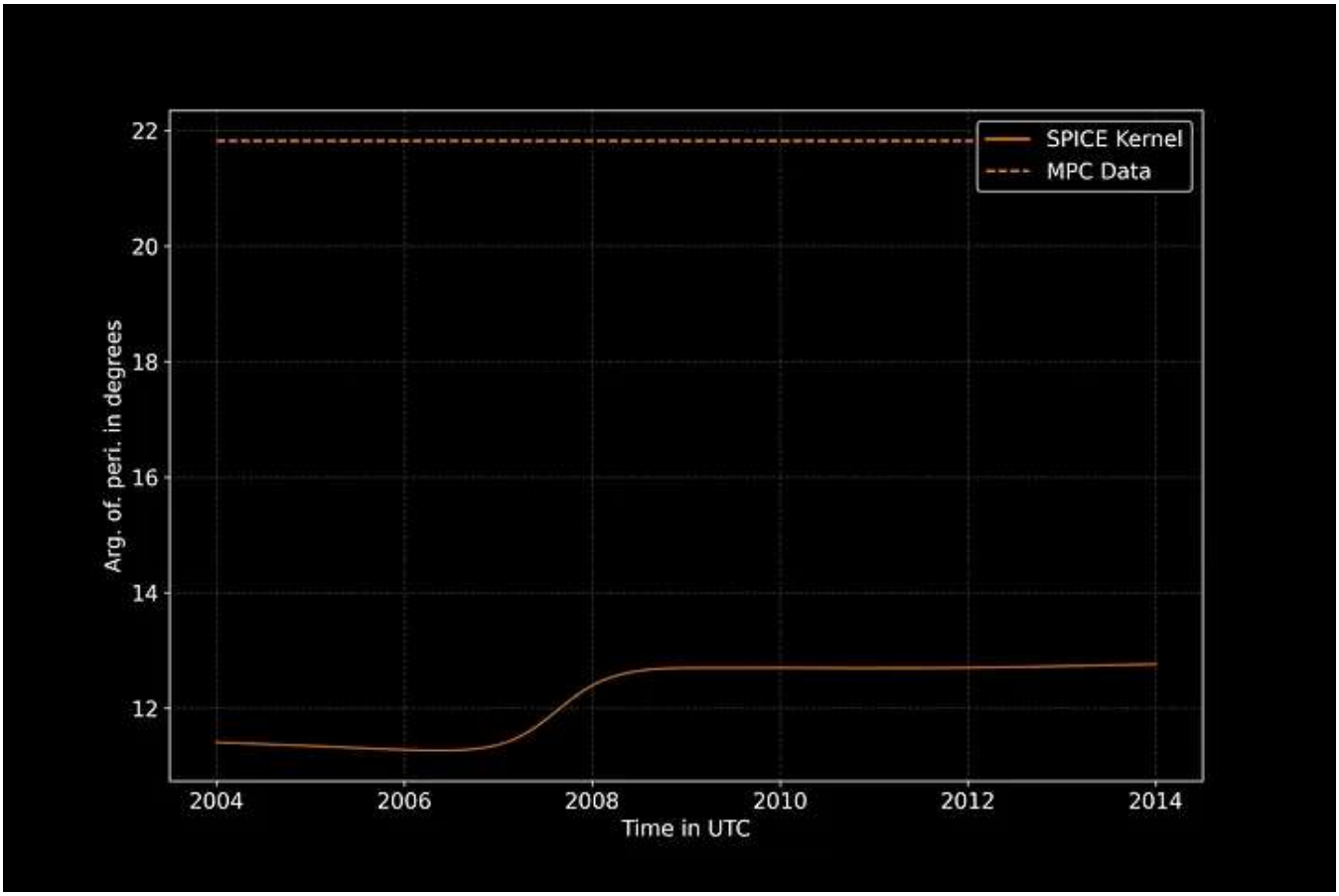


Perihelion in AU vs. the date-time in UTC. The solid line results from the SPICE kernel, the dashed line represents the static data from the MPC dataset. Credit: T. Albin



Eccentricity vs. the date-time in UTC. The solid line results from the SPICE kernel, the dashed line represents the static data from the MPC dataset. Credit: T. Albin

The last figure shows the argument of perihelion vs. the date-time in UTC. You can see that the perihelion's position of the comet rotated by around 2 degrees within 2 years. Is this "a lot"? Our home planet Earth is also affected by gravitational perturbations. However, due to its larger distance to other planets and its own mass the effects are rather small. The argument of perihelion of the Earth rotates by 360° in over 100,000 years [3]!



The argument of perihelion in degrees vs. the date-time in UTC. The solid line results from the SPICE kernel, the dashed line represents the static data from the MPC dataset. Credit: T. Albin

Conclusion

How shall one trust the data sources? Is there a way to get the "real" data? This cannot be answered. It always depends on your use case and required degree of detail. For large population analysis of all comets (Space Science with Python — Did we observe everything?), the MPC data are sufficient. Even for short-term predictions and trajectory computations, static results are fine. However, spacecraft mission design studies and detailed scientific analysis of particular comets require a more sophisticated and "corrected" view. Numerical analysis that consider gravitational perturbations and the outgassing of the comet are the most detailed ones. Provided SPICE kernels in the public repository are always a first good approach for good data quality.

But can the perturbations be explained quantitatively? Without too computational-heavy calculations? Yes! And this will be done next time, where we will use SPICE to derive our own solution of the history of 67P.

Thomas

Assignments

As some users suggested I will add small tasks at the end of every tutorial. That way, you can apply the things you learned. Feel free to contact me if you face any issues.

1. Does the Tisserand Parameter w.r.t. Jupiter change over time?
2. Visualise the distance between Jupiter and 67P between the initial and end time. Use *spkgps* for this purpose and think about the *targ* and *obs* parameters. Convert the results in AU.

Footnotes

How does one find out the NAIF IDs without checking the NAIF ID website? What are the time windows or coverages of the SPICE kernels? These questions can be answered with SPICE functions that are shown in a future tutorial, where I will provide a supplementary article on SPICE kernel information extraction.

References

[1] Michel C. Festou (Editor), H. Uwe Keller (Editor), Harold A. Weaver Jr. (Editor). (2004). *Comets II.* University of Arizona Press. pages 780. ISBN-10: 0816524505. ISBN-13: 978–0816524501

[2] Combi, M. R.; Harris, W. M.; Smyth, W. H. (2004). *Gas dynamics and kinetics in the cometary coma: theory and observations*. In: *Comets II* [1]. p. 523–552

[3] van den Heuvel, E. P. J. (1966). *On the Precession as a Cause of Pleistocene Variations of the Atlantic Ocean Water Temperatures*. Geophysical Journal International. 11: 323–336. Bibcode:1966GeoJ...11..323V. doi:10.1111/j.1365–246X.1966.tb03086.x

Python Space Science Programming Space Science With Python

Sign up for The Variable

By Towards Data Science

Every Thursday, the Variable delivers the very best of Towards Data Science: from hands-on tutorials and cutting-edge research to original features you don't want to miss. Take a look.

By signing up, you will create a Medium account if you don't already have one. Review our Privacy Policy for more information about our privacy practices.

Get this newsletter

About Help Terms Privacy

Get the Medium app

Download on the App Store

GET IT ON Google Play